

Бот для скрининга болезни Фабри

Документация разработчика

Версия 1.0 | 2025

1. Архитектура

1.1. Обзор

Приложение написано на Python 3.11. Единая точка входа — `bot/main.py`. Вся бизнес-логика вынесена в `bot/core.py`, который не имеет зависимостей от `aiogram` или `maxapi`.

1.2. Слои

Слой	Файл	Ответственность
Платформенный адаптер	<code>bot/main.py</code>	FSM, обработчики событий, keyboard-builders, пересылка в группу
Бизнес-логика	<code>bot/core.py</code>	STEPS, валидаторы, скоринг, генерация отчётов и PDF
Конфигурация скоринга	<code>bot/fabry_score_weights.json</code>	Веса ответов для вычисления индекса риска
Webhook-сервис	<code>webhook/webhook.py</code>	Дополнительный HTTP-эндпоинт (опционально)

1.3. Зависимости

Пакет	Версия	Назначение
<code>obabot</code>	<code>>=0.2.0</code>	Абстракция над Telegram (<code>aiogram</code>) и Max (<code>maxapi</code>)
<code>fpdf2</code>	<code>2.8.3</code>	Генерация PDF-отчётов
<code>python-dotenv</code>	<code>1.0.1</code>	Загрузка <code>.env</code>
<code>aiohhttp-socks</code>	<code>>=0.8.4</code>	SOCKS-прокси для Telegram

2. Структура проекта

```
fabri_bot/  
  .env          # токены и конфигурация  
  docker-compose.yml  
  Dockerfile  
  bot/  
    main.py      # точка входа  
    core.py      # бизнес-логика  
    fabry_score_weights.json # веса скоринга  
    requirements.txt  
  webhook/  
    webhook.py  
    Dockerfile.webhook  
  logs/
```

Важно: bot/main.py использует flat-импорт «from core import ...», поэтому запускать его нужно из директории bot/ или с PYTHONPATH=bot/. Команда «python -m bot.main» из корня не работает.

3. Модуль core.py

3.1. Модель шага (Step)

Датакласс Step описывает один вопрос анкеты:

Поле	Тип	Описание
key	str	Уникальный ключ ответа в словаре answers
kind	Literal['choice', 'text', 'collect']	Тип ввода
text	TextFn	Функция (answers -> str) для формирования текста вопроса
options	OptionsFn None	Функция (answers -> list[str]) для вариантов choice
condition	Condition	Предикат (data -> bool): показывать ли вопрос (по умолчанию True)
validator	Validator None	Функция (str, answers -> (bool, str)) для text-шагов

3.2. Список шагов STEPS

#	key	kind	Описание
0	role	choice	Кто заполняет: Пациент / Врач
1	sex	choice	Пол
2	age	text	Возраст (валидация 0–120)
3	fabry_confirmed	choice	Подтверждённый диагноз Фабри
4	relatives_fabry	choice	Родственники с болезнью Фабри
5	relatives_kidney_heart_stroke	choice	Родственники с почечными/сердечными/инсульт
6	pain_hands_feet	choice	Боли в ладонях и стопах
7	pain_triggers	choice	Кризисы Фабри (условие: pain != 'Никогда')
8	sweating	choice	Гипогидроз
9	gi_after_meals	choice	ЖКТ симптомы
10	early_satiety	choice	Быстрое насыщение
11	angiokeratomas	choice	Ангиокератомы
12	tachycardia	choice	Тахикардия
13	heart_enlargement	choice	ГКМП
14	dyspnea	choice	Одышка
15	myocardial_infarction	choice	Инфаркт в анамнезе
16	edema	choice	Отёки
17	proteinuria_creatinine	choice	Протеинурия / креатинин
18	chronic_kidney_disease	choice	ХБП
19	hearing_tinnitus	choice	Слух / тиннитус
20	dizziness	choice	Головокружение
21	eye_sign	choice	Офтальмологические признаки
22	stroke_tia_history	choice	Инсульт / ТИА

23	city	text	Город
24	specialization_position	text	Специализация (только врач)
25	workplace	text	Место работы (только врач)
26	additional_info	collect	Доп. материалы (только пациент)
27	callback_pref	choice	Запрос обратного звонка (только пациент)
28	sms_pref	choice	Запрос СМС (только пациент, если не выбран звонок)
29	full_name	text	ФИО
30	phone	text	Телефон (условие: врач или выбран звонок/СМС)

4. Конечный автомат (FSM)

4.1. Состояния SurveyFSM

Состояние	Когда активно
waiting_consent	Ожидание нажатия кнопки согласия/отказа
waiting_choice	Ожидание нажатия инлайн-кнопки ответа
waiting_text	Ожидание текстового сообщения
collecting_additional	Ожидание доп. материалов (файлы, фото, текст)

4.2. Переходы между состояниями

- /start → waiting_consent
- consent|yes → waiting_choice (шаг role)
- consent|no → завершение (без FSM)
- ans|<idx>|<opt> → next_step_index() → waiting_choice / waiting_text / collecting_additional / финал
- Текстовый ввод → валидация → следующий шаг
- collect_done|<idx> → next_step_index() → следующий шаг / финал

4.3. Навигация по шагам

Функции `next_step_index(start_from, data)` и `step_by_index(idx)` из `core.py` определяют следующий применимый шаг. Условие `condition` каждого шага вычисляется на текущем состоянии словаря `data`, что обеспечивает динамическое ветвление.

5. Система скоринга

5.1. Файл весов fabry_score_weights.json

JSON-объект верхнего уровня: ключи — key шага, значения — объект {ответ: вес}.

Ключи и значения, начинающиеся с «_», игнорируются (комментарии).

Пример:

```
"pain_hands_feet": { "Да": 2.0, "Никогда": 0 }
```

```
"angiokeratomas": { "Между пупком и коленями": 3.0, "Нет, не замечал(а)": 0 }
```

5.2. Функции скоринга

Функция	Сигнатура	Возвращает
calculate_fabry_score_details	(answers) -> (float, list[dict])	Балл + детальная разбивка
calculate_fabry_score	(answers) -> float	Только итоговый балл
get_score_interpretation	(score: float) -> str	«Риск выявлен» или «Нет индикаторов риска»

6. Генерация PDF

6.1. Функция `generate_pdf_report`

`generate_pdf_report(data: dict) -> bytes`

Принимает FSM-данные сессии. Возвращает PDF в виде байтов. Шрифт — DeJaVuSans.ttf (ищется в нескольких системных путях). Структура PDF: заголовок → оценка риска → сработавший скоринг → блоки ответов в виде таблиц.

6.2. Поиск шрифта `_find_dejavu_font()`

- `/usr/share/fonts/truetype/dejavu/DejaVuSans.ttf`
- `/usr/share/fonts/TTF/DejaVuSans.ttf`
- `/System/Library/Fonts/Supplemental/Arial Unicode.ttf`
- `bot/DejaVuSans.ttf` (рядом со скриптом)

При отсутствии шрифта функция выбрасывает `FileNotFoundError`. Для Docker-образа установите пакет `fonts-dejavu-core`.

7. Конфигурация через .env

Переменная	По умолчанию	Описание
BOT_TOKEN	—	Telegram Bot API token
MAX_BOT_TOKEN	—	Max Bot API token
GROUP_CHAT_ID	—	Telegram-группа для пересылки анкет
MAX_GROUP_CHAT_ID	—	Max-группа для пересылки анкет
LOG_CHAT_ID	=GROUP_CHAT_ID	Telegram-чат для логов приложения
HOTLINE_PHONE	[Phone8]	Номер горячей линии
CONSENT_DECLINE_PHONE	=HOTLINE_PHONE	Телефон при отказе от согласия
TG_PROXY	—	URL прокси для Telegram
TEST_MODE	0	1 — использовать TEST_BOT_TOKEN и TEST_GROUP_CHAT_ID
TEST_BOT_TOKEN	—	Тестовый Telegram-токен
TEST_GROUP_CHAT_ID	—	Тестовая группа

8. Запуск и развёртывание

8.1. Локальный запуск

```
# Создать виртуальное окружение
python3.11 -m venv .venv
source .venv/bin/activate # Windows: .venv\Scripts\activate
```

```
# Установить зависимости
pip install -r bot/requirements.txt
```

```
# Запустить бот (из директории bot/)
cd bot
python main.py
```

8.2. Docker Compose

```
docker compose up -d # запустить
docker compose logs -f # смотреть логи
docker compose down # остановить
```

8.3. Проверка синтаксиса (без тест-сьюта)

```
python -m py_compile bot/main.py bot/core.py
```

8.4. Systemd-сервис

```
[Service]
WorkingDirectory=/opt/fabri_bot/bot
EnvironmentFile=/opt/fabri_bot/.env
ExecStart=/opt/fabri_bot/.venv/bin/python main.py
Restart=on-failure
```

9. Валидаторы

Функция	Проверяет	Ошибка при
validate_age	Возраст 0–120	Не цифра или вне диапазона
validate_nonempty	Непустой текст	Пустая строка или только пробелы
validate_full_name	ФИО (буквы/дефис/точка, длина ≥ 2)	Цифры или недопустимые символы
validate_phone	Телефон 10–15 цифр	Недопустимые символы или длина

Сигнатура всех валидаторов: (text: str, answers: dict) -> tuple[bool, str]

bool — признак успеха, str — текст ошибки (пустая строка при успехе).

10. Клавиатуры (main.py)

Функция	Тип	Назначение
consent_keyboard()	InlineKeyboardBuilder	Согласие / Отказ / Горячая линия
choice_keyboard(step_index, options)	InlineKeyboardBuilder	Варианты ответа + Горячая линия
text_keyboard()	InlineKeyboardBuilder	Только кнопка Горячей линии
collect_keyboard(step_index)	InlineKeyboardBuilder	«Продолжить» + Горячая линия
phone_reply_keyboard()	ReplyKeyboardMarkup	Кнопка «Поделиться номером»

11. Расширение анкеты

Чтобы добавить новый вопрос:

1. Добавьте функции `_step_text_XXX` и (при необходимости) `_step_opts_XXX` в `core.py`.
2. Добавьте `Step(...)` в список `STEPS` в нужную позицию.
3. Добавьте метку в `QUESTION_LABELS`.
4. Добавьте ключ в нужный блок `NOSOLOGY_BLOCKS`.
5. Если вопрос влияет на скоринг — добавьте весовые коэффициенты в `fabry_score_weights.json`.

Условные шаги:

Параметр `condition` принимает `callable(data: dict) -> bool`.

Если `condition` вернул `False` — шаг пропускается.

Пример: `condition=lambda d: d.get('answers', {}).get('pain_hands_feet') != 'Никогда'`